

Master's Thesis

Adaptive Rounding for Efficient Post-Training Quantization of Deep Neural Networks

48246347 Piye Liu

Supervisor Prof. Hiroshi Nakamura

August, 2026

Department of Information Physics and Computing
Graduate School of Information Science and Technology
The University of Tokyo

概要

Abstract in Japanese should be written here.

Abstract

300 words abstract in English should be written here.

Contents

Chapter 1	Introduction	1
Chapter 2	Background	2
2.1	Compressibility of Deep Neural Networks	2
2.2	Baseline Round-to-Nearest Quantization	2
2.3	Quantization is Weight Perturbation	2
2.4	Small Language Models	2
2.5	Effects of Weight Perturbation	3
Chapter 3	Literature Review	4
Chapter 4	Proposed Method	5
4.1	AdaRound with Gradient Awareness	5
Chapter 5	Evaluation	7
Chapter 6	Conclusion	8
	Bibliography	9
A	Appendix	10

Chapter 1

Introduction

Chapter 2

Background

Source coding theorem
+ MLE estimation of
Qwen3's entropy here

2.1 Compressibility of Deep Neural Networks

2.2 Baseline Round-to-Nearest Quantization

The baseline for post-training quantization is symmetric uniform quantization using round-to-nearest (RTN) rounding. [1]

Algorithm 1 Baseline RTN Quantization

Require: W (unquantized weights), b (target bit width)

Ensure: Quantized model W_q

- 1: $q_{\min} \leftarrow -2^{b-1}$
 - 2: $q_{\max} \leftarrow 2^{b-1} - 1$
 - 3: $\text{scale} \leftarrow \frac{\max(W)}{q_{\max}}$
 - 4: $W_q \leftarrow \text{clamp}(\lceil \frac{W}{\text{scale}} \rceil, q_{\min}, q_{\max})$
 - 5: **return** W_q, scale
-

Algorithm 2 Baseline RTN Dequantization

Require: W_q (quantized weights), scale

Ensure: Dequantized weights $\hat{W}$

- 1: $\hat{W} \leftarrow W_q \cdot \text{scale}$
 - 2: **return** $\hat{W}$
-

2.3 Quantization is Weight Perturbation

2.4 Small Language Models

In this section, we will examine TinyStories [?], a work that pushes the boundaries of small language models, and provide a few custom language models for further experimentation. By using a low-variance synthetic dataset (children's stories) generated by GPT-3.5 and GPT-4 containing only words a typical 3-4 year old would understand, they were able to generate coherent output using models with as few as 1 million parameters.

One limitation we observed in this work is their usage of the GPT-Neo [?] tokenizer, restricted to the top 10,000 tokens. Using a generic tokenizer for a low-vocabulary dataset

like TinyStories might result in wasted weights (tokens that never appear in TinyStories), and might not provide the optimal tokenization for our specialized dataset.

As a preliminary experiment, we train two small language models on TinyStories v1 and TinyStories v2, respectively. The original paper only provided models trained on TinyStories v1. We then evaluate their accuracy using perplexity on the validation datasets as a metric.

Table 2.1: Custom Language Models

	Reference	Ours (TinyStories v1)	Ours (TinyStories v2)
Architecture	GPT-Neo	LLaMa	LLaMa
Parameter Count	1.17M	0.57M	0.46M
Training Data	TinyStories v1		TinyStories v2
Tokenizer (Vocab Size)	GPT-Neo	Custom	Custom
Vocab Size	10000	1024	1024
Perplexity (TinyStories v1)	19.3919	11.9006	—
Perplexity (TinyStories v2)	—	—	8.0994

2.5 Effects of Weight Perturbation

Chapter 3

Literature Review

Chapter 4

Proposed Method

4.1 AdaRound with Gradient Awareness

To reconcile AdaRound’s assumption that the gradient term in ?? can be ignored, and Central Flows’ evidence that the gradient is nonzero even for pretrained converged models, we propose a modified version of AdaRound to incorporate awareness of the local gradient.

In order to minimize,

$$\mathbb{E} \left[g^{(w)^T} \Delta W \right], \quad (4.1)$$

we can attempt to force ΔW to have the same sign as $-g^{(w)^T}$. We approximate this by finetuning a model on a small amount of training data, and compute the Exponential Moving Average (EMA) of the model during each iteration. We then use $\text{sign}(W'_{\text{avg}} - W)$, the sign of the difference between the weights before and after finetuning, as a proxy to $-\text{sign}(g^{(w)^T})$. Finally, we add the Binary Cross Entropy loss between the current rounding directions and $\text{sign}(W'_{\text{avg}} - W)$ as a penalty term to AdaRound’s optimization objective.

Algorithm 3 Proposed Method (Finetuning)

Require: W , X (calibration data), η (learning rate), B (batch size)

Ensure: Finetuned model W' , EMA of models during finetuning W'_{avg}

```

1:  $W' \leftarrow W$ 
2:  $W'_{\text{avg}} \leftarrow W$ 
3:  $N \leftarrow \frac{X}{B}$  ▷ Number of Iterations
4:  $\alpha \leftarrow \frac{2}{N+1}$  ▷ EMA Smoothing Factor
5: for each batch  $x_b$  in  $X$  do
6:    $\hat{x}_b \leftarrow W'(x_b)$  ▷ Forward pass
7:    $\mathcal{L} \leftarrow \text{CrossEntropyLoss}(\hat{x}_b, x_b)$ 
8:    $W' \leftarrow W' - \eta \cdot \nabla \mathcal{L}$  ▷ Backward pass
9:    $W'_{\text{avg}} \leftarrow \alpha \cdot W'_{\text{avg}} + (1 - \alpha) \cdot W'$ 
10: end for
11: return  $W'$ ,  $W'_{\text{avg}}$ 

```

Algorithm 4 Proposed Method (Calibration)

Require: W, W'_{avg}, X (calibration data), η (learning rate), t (number of iterations)

Ensure: Quantized model W_q

1: (TODO)

2: **return** W_q

Chapter 5

Evaluation

Chapter 6

Conclusion

Bibliography

- [1] Markus Nagel, Marios Fournarakis, Rana Ali Amjad, Yelysei Bondarenko, Mart van Baalen, and Tijmen Blankevoort. A White Paper on Neural Network Quantization, June 2021.

A

Appendix